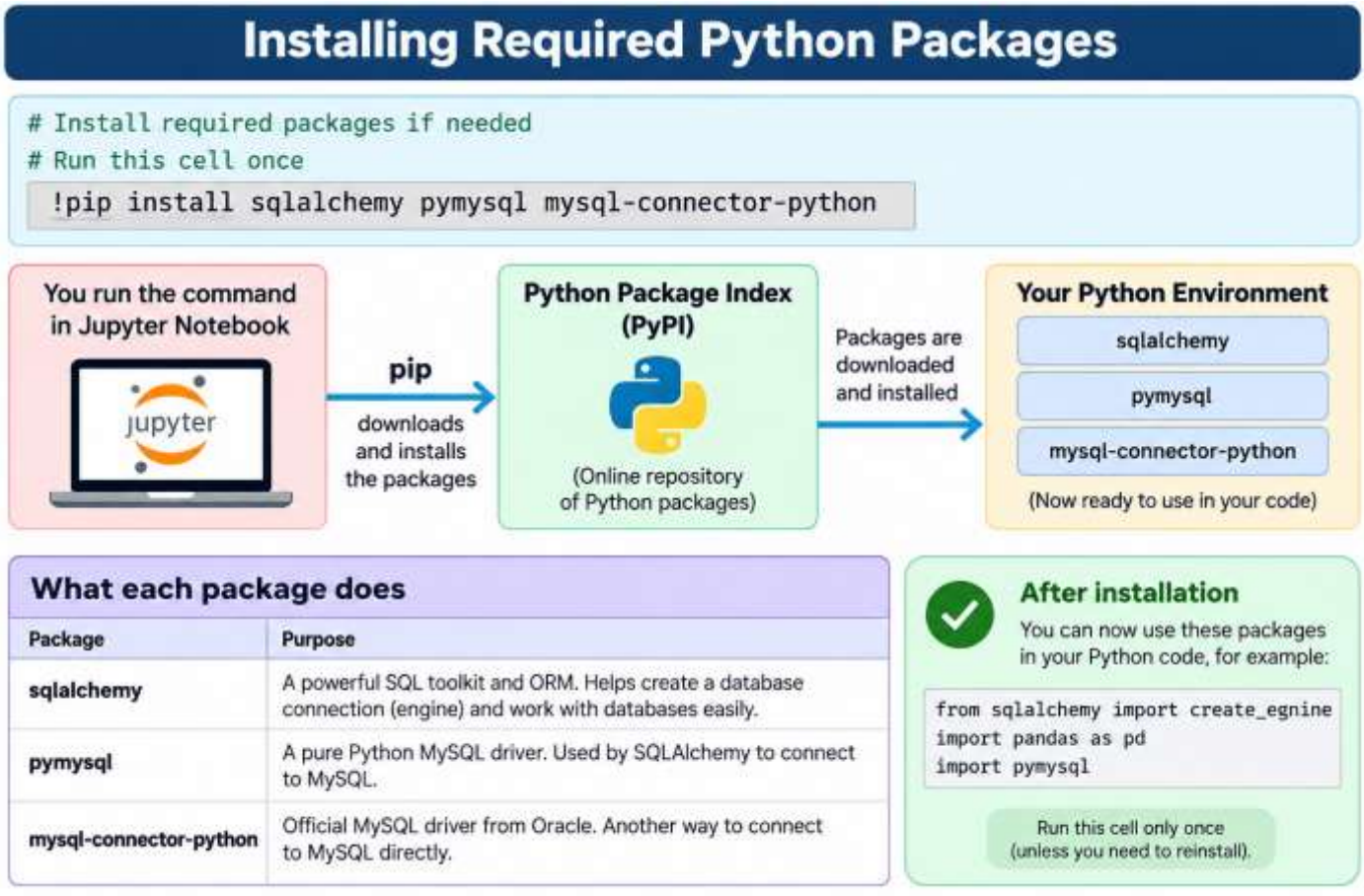
Data Engineering/Data Analytics

Day-5

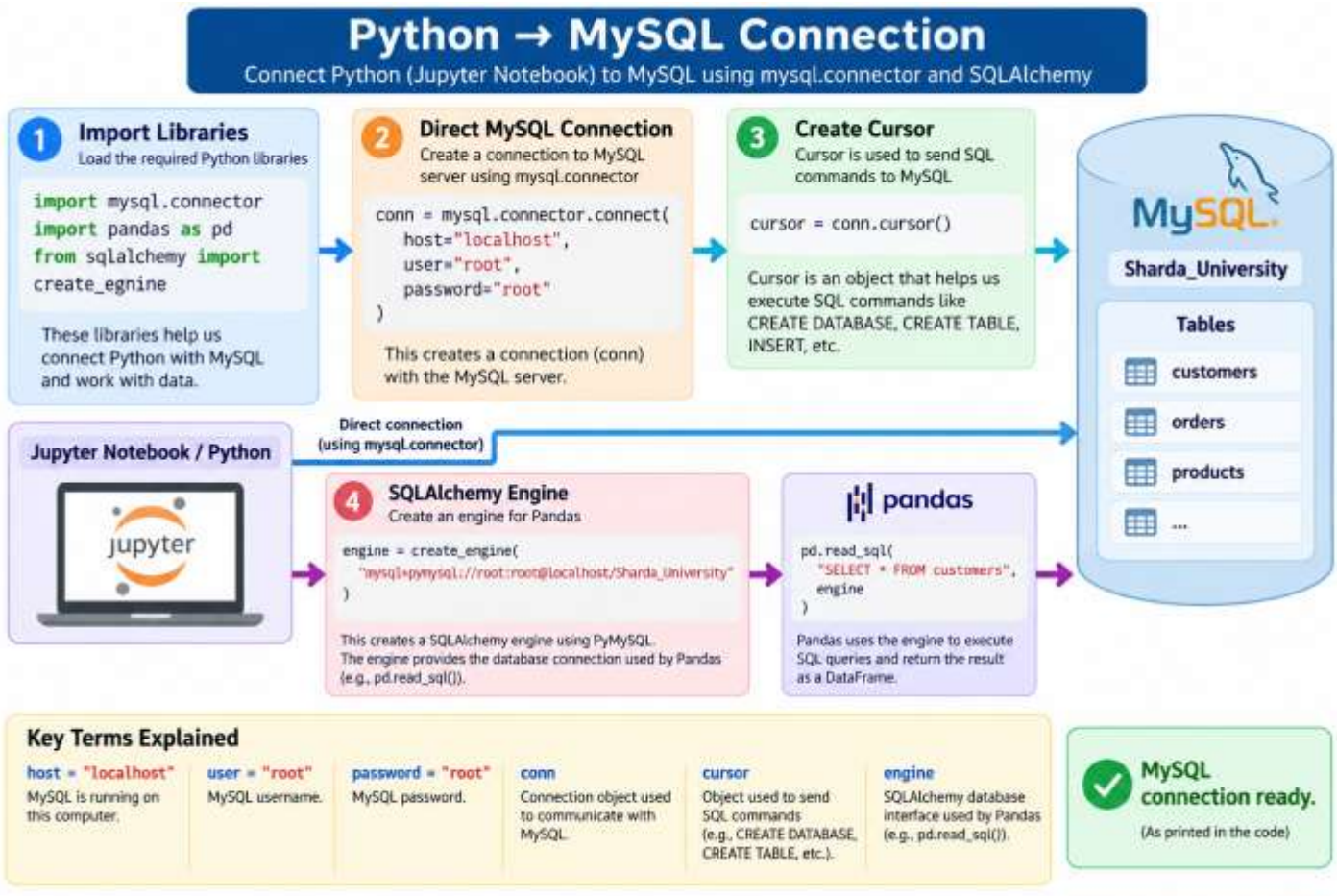
```
# Install required packages if needed
# Run this cell once
!pip install sqlalchemy pymysql mysql-connector-python
```



1. Import Libraries and Create MySQL Connection

```
]: import mysql.connector
import pandas as pd
from sqlalchemy import create_engine

# Connection for database/table creation
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="root"
)

cursor = conn.cursor()

# SQLAlchemy engine for Pandas
engine = create_engine(
    "mysql+pymysql://root:root@localhost/Sharda_University"
)

print("MySQL connection ready.")
```

```
MySQL connection ready.
```



This cell does **two types of database setup**:

1. Creates a direct MySQL connection using `mysql.connector`.
2. Creates a SQLAlchemy `engine` that Pandas can use to run SQL queries.

Object	Main use
conn	Direct MySQL operations
cursor	Execute SQL commands such as `CREATE`, `INSERT`, etc.
engine	Used by Pandas/SQLAlchemy to run queries
pd.read_sql()	Reads SQL results into a Pandas DataFrame

Easy way to remember

cursor = send commands

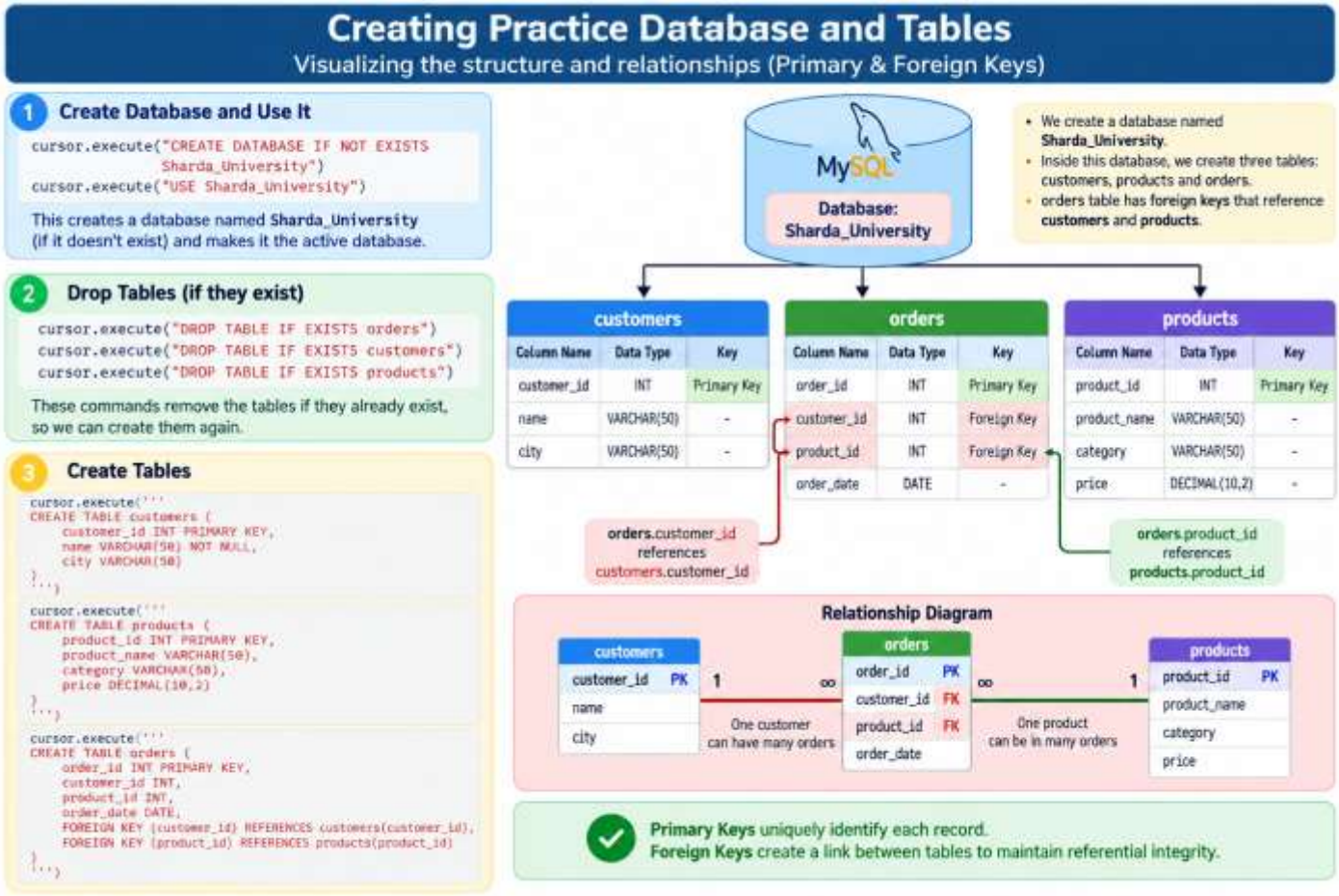
engine = connect Pandas to database

conn = direct MySQL connection

pd.read_sql() = bring SQL result into Pandas

2. Create Practice Database and Tables

```
# When Python connects to MySQL, we need an object that can send SQL commands to MySQL.
# That object is called a cursor.
#cursor is used to execute SQL statements and retrieve results from the database.
#execute() is a method of the cursor.
#Python tells the cursor to send USE Sharda_University to MySQL. MySQL then executes that command

cursor.execute("CREATE DATABASE IF NOT EXISTS Sharda_University")
cursor.execute("USE Sharda_University")

cursor.execute("DROP TABLE IF EXISTS orders")
cursor.execute("DROP TABLE IF EXISTS customers")
cursor.execute("DROP TABLE IF EXISTS products")

#''' in Python means triple single quotes.
#It is mainly used to create a multi-line string.
cursor.execute('''
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    city VARCHAR(50)
)
''')

cursor.execute('''
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(50),
```



3. SELECT, WHERE, DISTINCT, ORDER BY and LIMIT

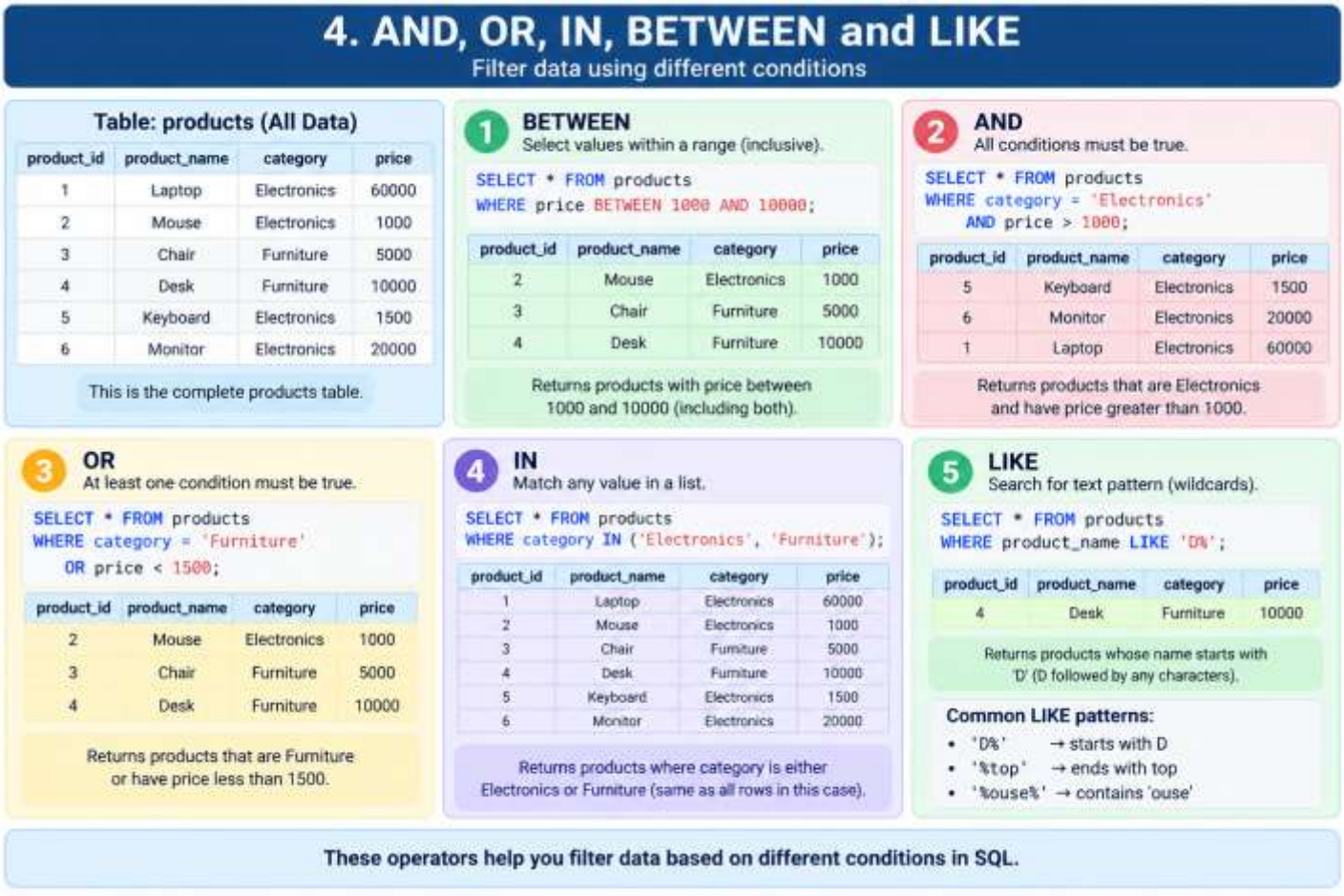
```
#Pandas, run this SQL query using this MySQL database connection (engine) and give  the result as a DataFrame
#Send this SQL query to MySQL through the SQLAlchemy engine and return the result in Pandas.
# SELECT
pd.read_sql("SELECT * FROM customers", engine)
```

	customer_id	name	city
0	1	Amit	Delhi
1	2	Neha	Mumbai
2	3	Ravi	Delhi
3	4	Priya	Pune
4	5	John	Delhi

4. AND, OR, IN, BETWEEN and LIKE

```
pd.read_sql("""
SELECT * FROM products
WHERE price BETWEEN 1000 AND 10000
""", engine)
```

	product_id	product_name	category	price
0	2	Mouse	Electronics	1000.0
1	3	Chair	Furniture	5000.0
2	4	Desk	Furniture	10000.0

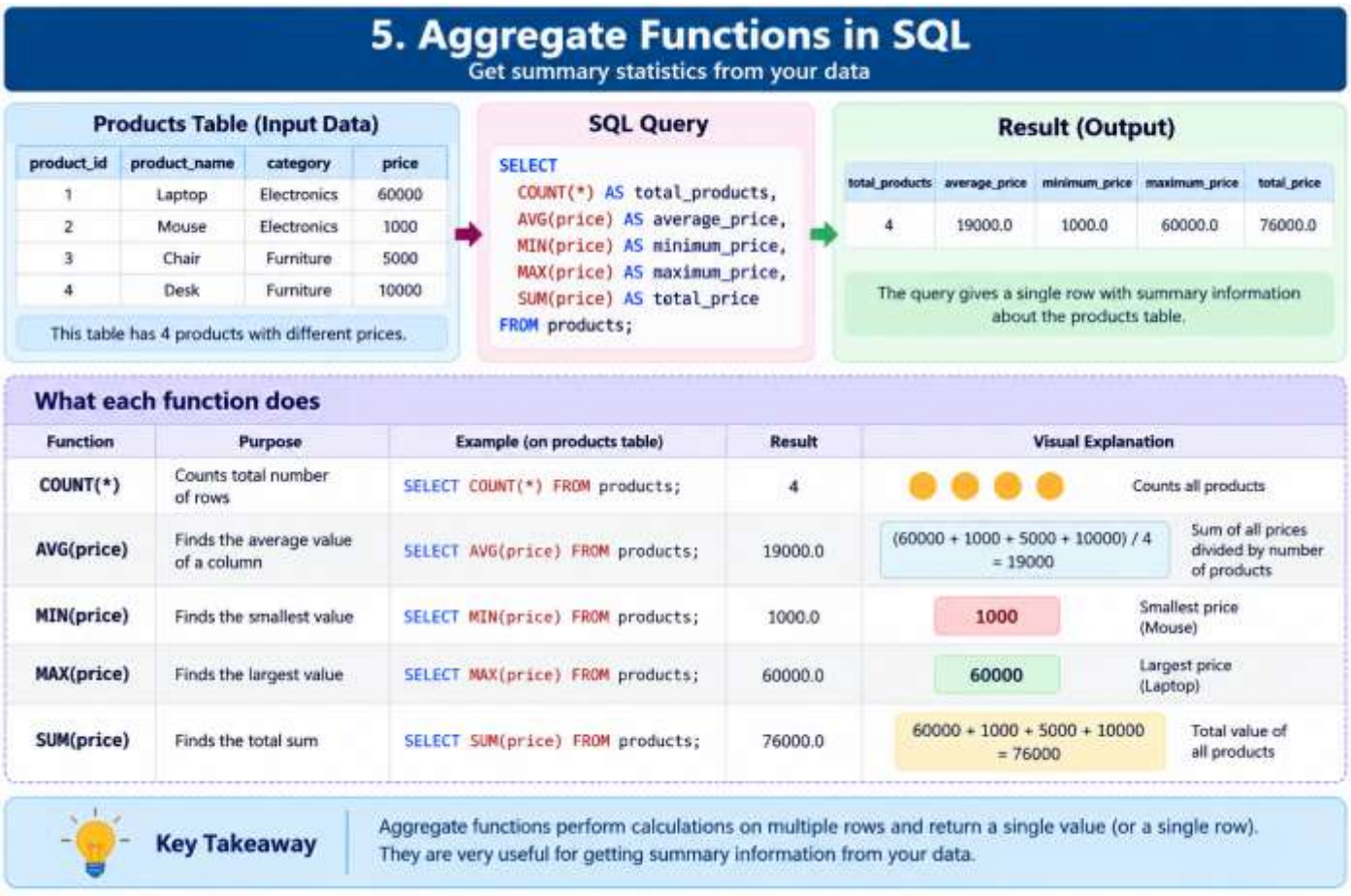


5. Aggregate Functions

[42]:
```
pd.read_sql("""
SELECT
    COUNT(*) AS total_products,
    AVG(price) AS average_price,
    MIN(price) AS minimum_price,
    MAX(price) AS maximum_price,
    SUM(price) AS total_price
FROM products
""", engine)
```

[42]:

	total_products	average_price	minimum_price	maximum_price	total_price
0	4	19000.0	1000.0	60000.0	76000.0



WHERE vs HAVING

- `WHERE` filters rows before grouping.
- `HAVING` filters groups after `GROUP BY`.

[43]:
```
pd.read_sql("""
SELECT category, AVG(price) AS avg_price
FROM products
GROUP BY category
HAVING AVG(price) > 5000
""", engine)
```

[43]:

	category	avg_price
0	Electronics	30500.0
1	Furniture	7500.0

WHERE vs HAVING

Both are used to filter data, but they work at different stages in the query.

1. Original Data (products table)

product_id	product_name	category	price
1	Laptop	Electronics	60000
2	Mouse	Electronics	1000
3	Chair	Furniture	5000
4	Desk	Furniture	10000
5	Keyboard	Electronics	1500
6	Monitor	Electronics	20000

This is the complete data in the products table.

2 WHERE (Row Filtering)

- Filters individual rows **before** grouping.
- Works on column values.
- Does not use aggregate functions (e.g., AVG, SUM).

Syntax:

```
SELECT columns
FROM table
WHERE condition;
```

3 HAVING (Group Filtering)

- Filters groups **after** GROUP BY.
- Works on aggregate functions.
- Used with GROUP BY.

Syntax:

```
SELECT columns, AGG_FUNCTION()
FROM table
GROUP BY columns
HAVING condition;
```

Example 1: Using WHERE

Get products with price > 5000 (row-level filter)

```
SELECT * FROM products
WHERE price > 5000;
```

product_id	product_name	category	price
1	Laptop	Electronics	60000
4	Desk	Furniture	10000
6	Monitor	Electronics	20000

WHERE filters rows before grouping.

Example 2: Using HAVING

Get categories with average price > 5000

```
SELECT category, AVG(price) AS avg_price
FROM products
GROUP BY category
HAVING AVG(price) > 5000;
```

category	avg_price
Electronics	30500.0
Furniture	7500.0

HAVING filters groups after GROUP BY using an aggregate function (AVG).

Step-by-Step Process (for HAVING example)

1. Read all rows from products table. — 6 rows
2. Group by **category** and calculate average price. — 3 groups

category	avg_price
Electronics	30500.0
Furniture	7500.0
Toys	500.0

3. Apply HAVING AVG(price) > 5000. — 2 groups

category	avg_price
Electronics	30500.0
Furniture	7500.0

Key Differences

Aspect	WHERE	HAVING
When it is applied	Before grouping	After grouping
What it filters	Individual rows	Grouped rows
Can use aggregate functions	No	Yes (e.g., AVG, SUM, COUNT)
Used with GROUP BY	Not required	Required
Example condition	price > 5000	AVG(price) > 5000

Key Takeaway

- Use **WHERE** to filter rows based on a condition on column values.
- Use **HAVING** to filter groups based on a condition on aggregate values.
- WHERE comes before GROUP BY, and HAVING comes after GROUP BY in the logical order of SQL execution.

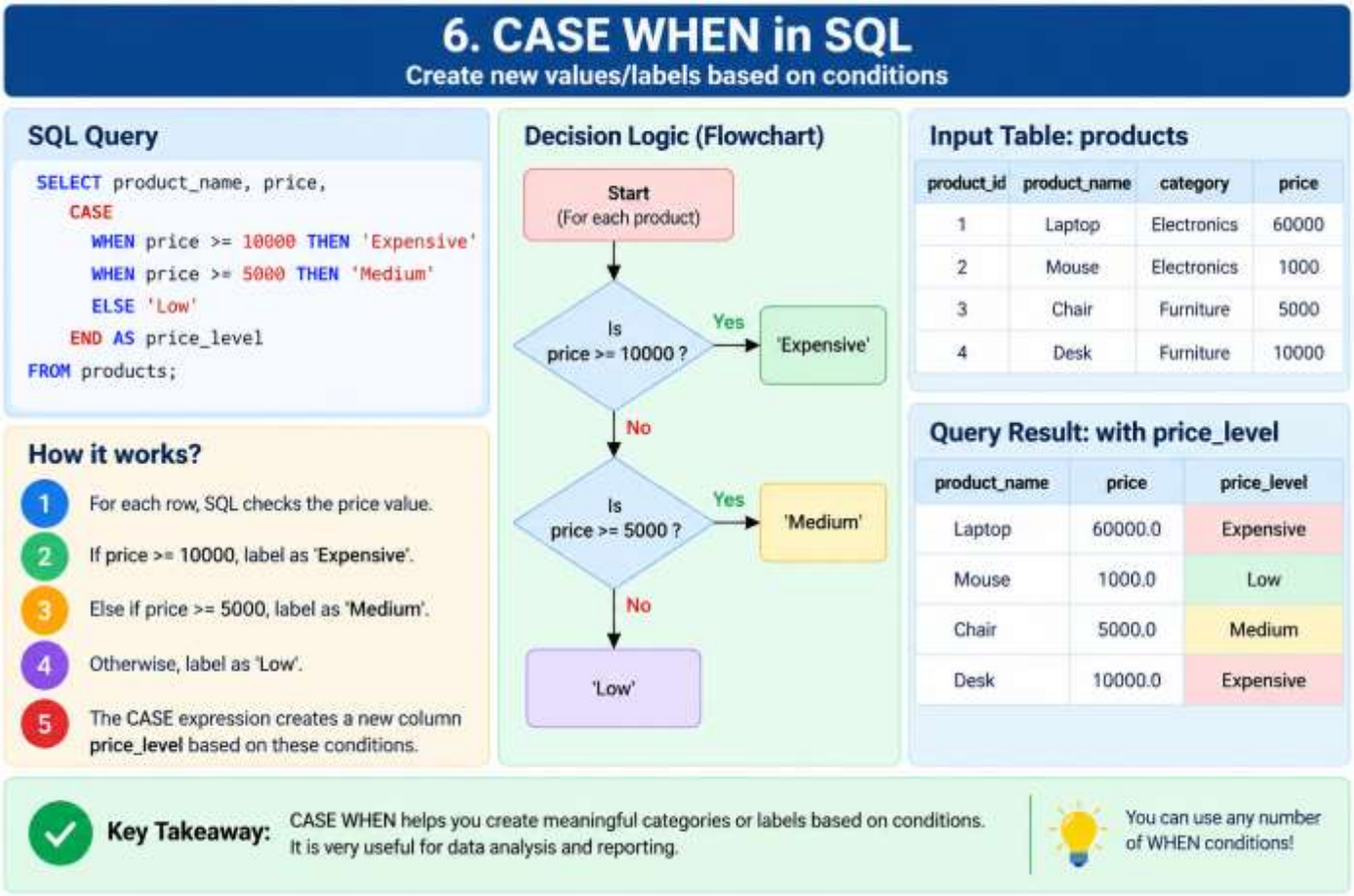
WHERE filters data, **HAVING** filters **summaries**.

6. CASE WHEN

4]:
```
pd.read_sql("""
SELECT product_name, price,
       CASE
           WHEN price >= 10000 THEN 'Expensive'
           WHEN price >= 5000 THEN 'Medium'
           ELSE 'Low'
       END AS price_level
FROM products
""", engine)
```

4]:

	product_name	price	price_level
0	Laptop	60000.0	Expensive
1	Mouse	1000.0	Low
2	Chair	5000.0	Medium
3	Desk	10000.0	Expensive



7. String and Date Functions

[45]:
```
pd.read_sql("""
SELECT name,
       UPPER(name) AS upper_name,
       LENGTH(name) AS name_length
FROM customers
""", engine)
```

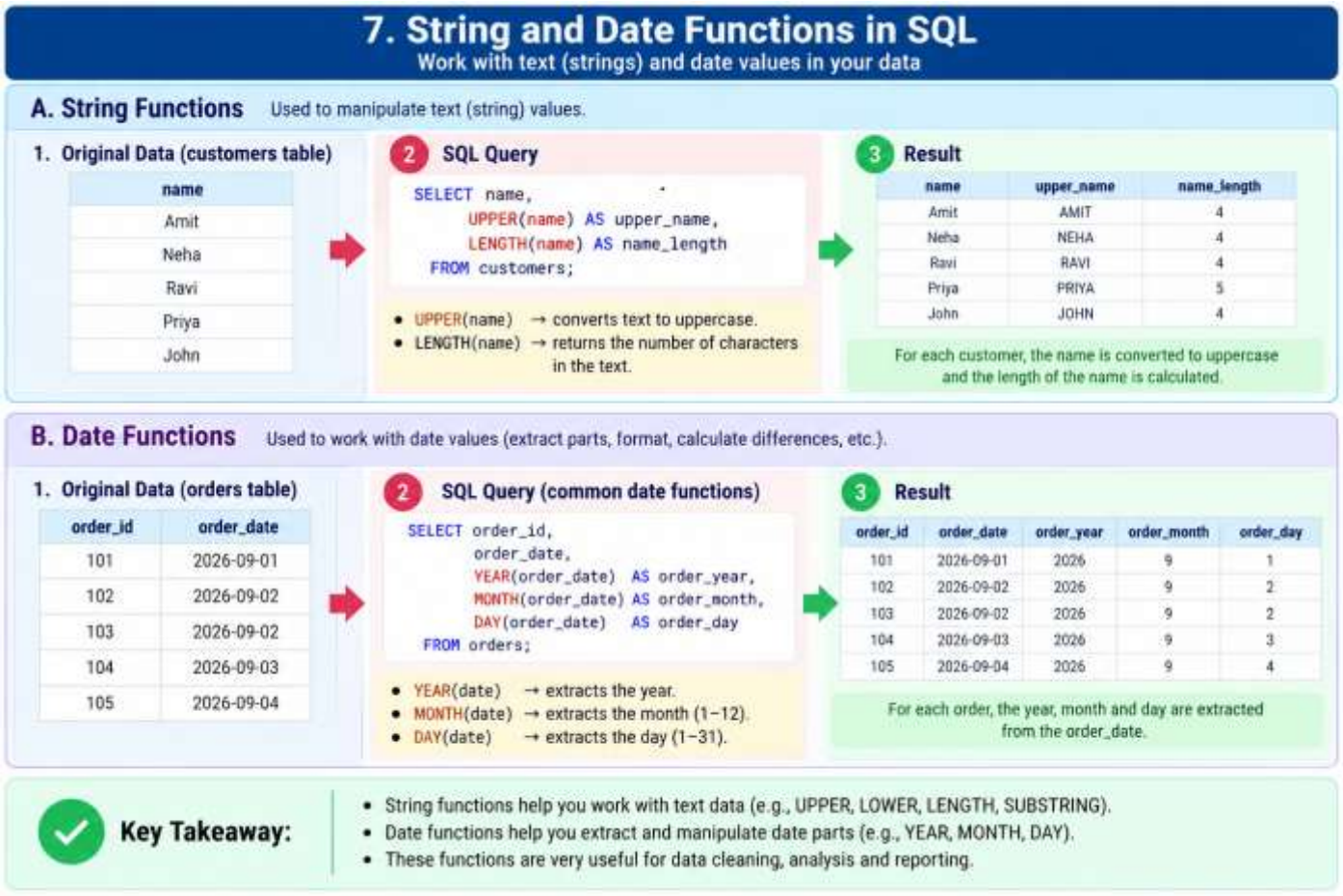
[45]:

	name	upper_name	name_length
0	Amit	AMIT	4
1	Neha	NEHA	4
2	Ravi	RAVI	4
3	Priya	PRIYA	5
4	John	JOHN	4

[46]:
```
pd.read_sql("""
SELECT order_id, order_date,
       YEAR(order_date) AS order_year,
       MONTH(order_date) AS order_month
FROM orders
""", engine)
```

[46]:

	order_id	order_date	order_year	order_month
0	101	2026-09-01	2026	9
1	102	2026-09-02	2026	9
2	103	2026-09-02	2026	9
3	104	2026-09-03	2026	9
4	105	2026-09-04	2026	9

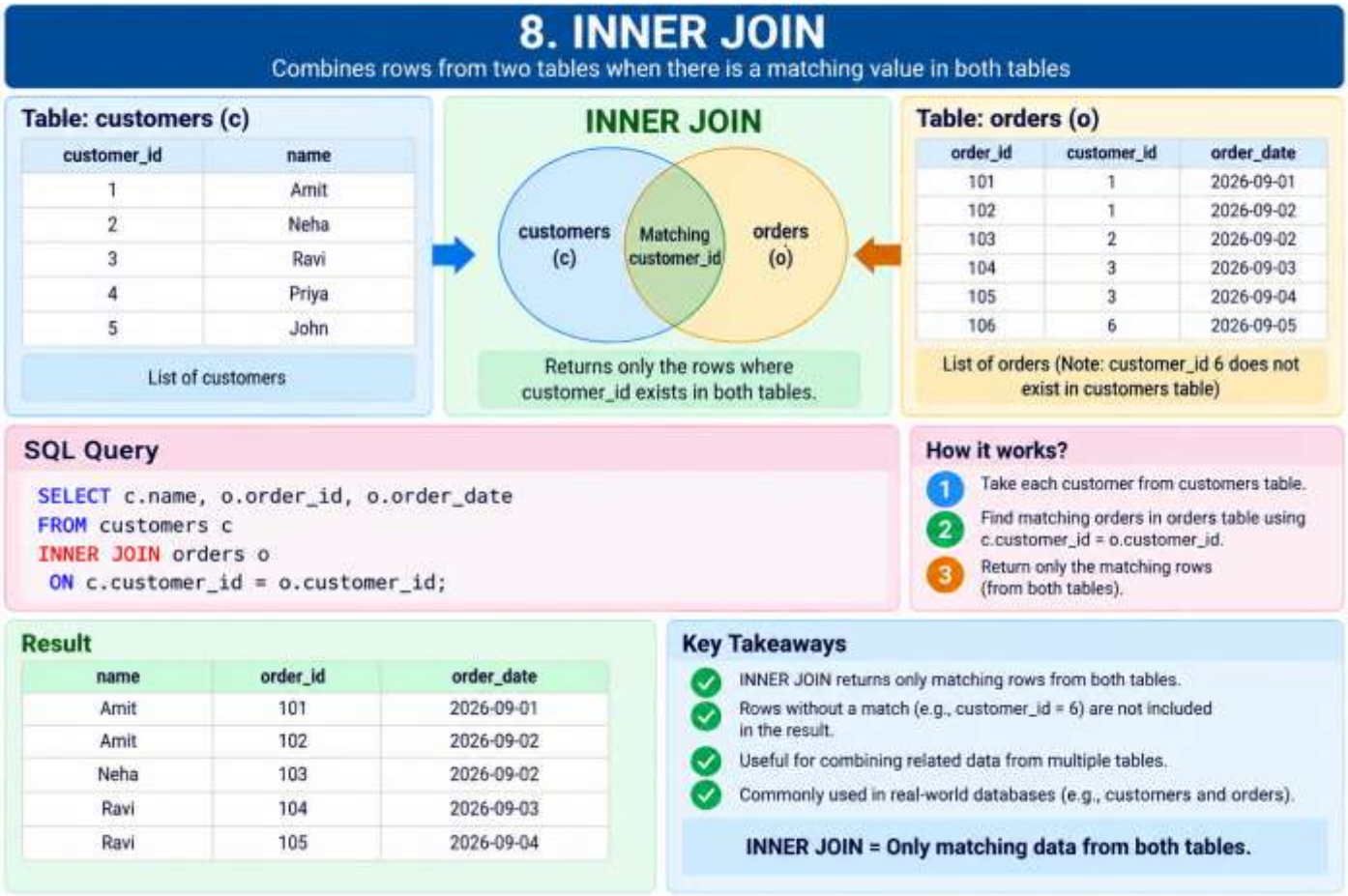


8. INNER JOIN

[47]:
```
pd.read_sql("""
SELECT c.name, o.order_id, o.order_date
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id
""", engine)
```

[47]:

	name	order_id	order_date
0	Amit	101	2026-09-01
1	Amit	102	2026-09-02
2	Neha	103	2026-09-02
3	Ravi	104	2026-09-03
4	Ravi	105	2026-09-04



9. JOIN Three Tables

[48]:
```
pd.read_sql("""
SELECT c.name, p.product_name, o.quantity
FROM orders o
JOIN customers c
    ON o.customer_id = c.customer_id
JOIN products p
    ON o.product_id = p.product_id
""", engine)
```

[48]:

	name	product_name	quantity
0	Amit	Laptop	1
1	Amit	Mouse	2
2	Ravi	Mouse	3
3	Neha	Chair	1
4	Ravi	Desk	1

9. JOIN Three Tables

Combine data from multiple related tables to get a complete view

1. customers (c)

Customer details

customer_id	name
1	Amit
2	Neha
3	Ravi
4	Priya
5	John

1 ∞

c.customer_id = o.customer_id

2. orders (o)

Which customer placed an order

order_id	customer_id	order_date
101	1	2026-09-01
102	1	2026-09-01
103	3	2026-09-02
104	2	2026-09-02
105	3	2026-09-03

∞ 1

o.product_id = p.product_id

3. products (p)

Product details

product_id	product_name
1	Laptop
2	Mouse
3	Chair
4	Desk

SQL Query (Join Three Tables)

```
SELECT c.name, p.product_name, o.quantity
FROM orders o
JOIN customers c
     ON o.customer_id = c.customer_id
JOIN products p
     ON o.product_id = p.product_id;
```

Result (Joined Data)

	name	product_name	quantity
0	Amit	Laptop	1
1	Amit	Mouse	2
2	Ravi	Mouse	3
3	Neha	Chair	1
4	Ravi	Desk	1

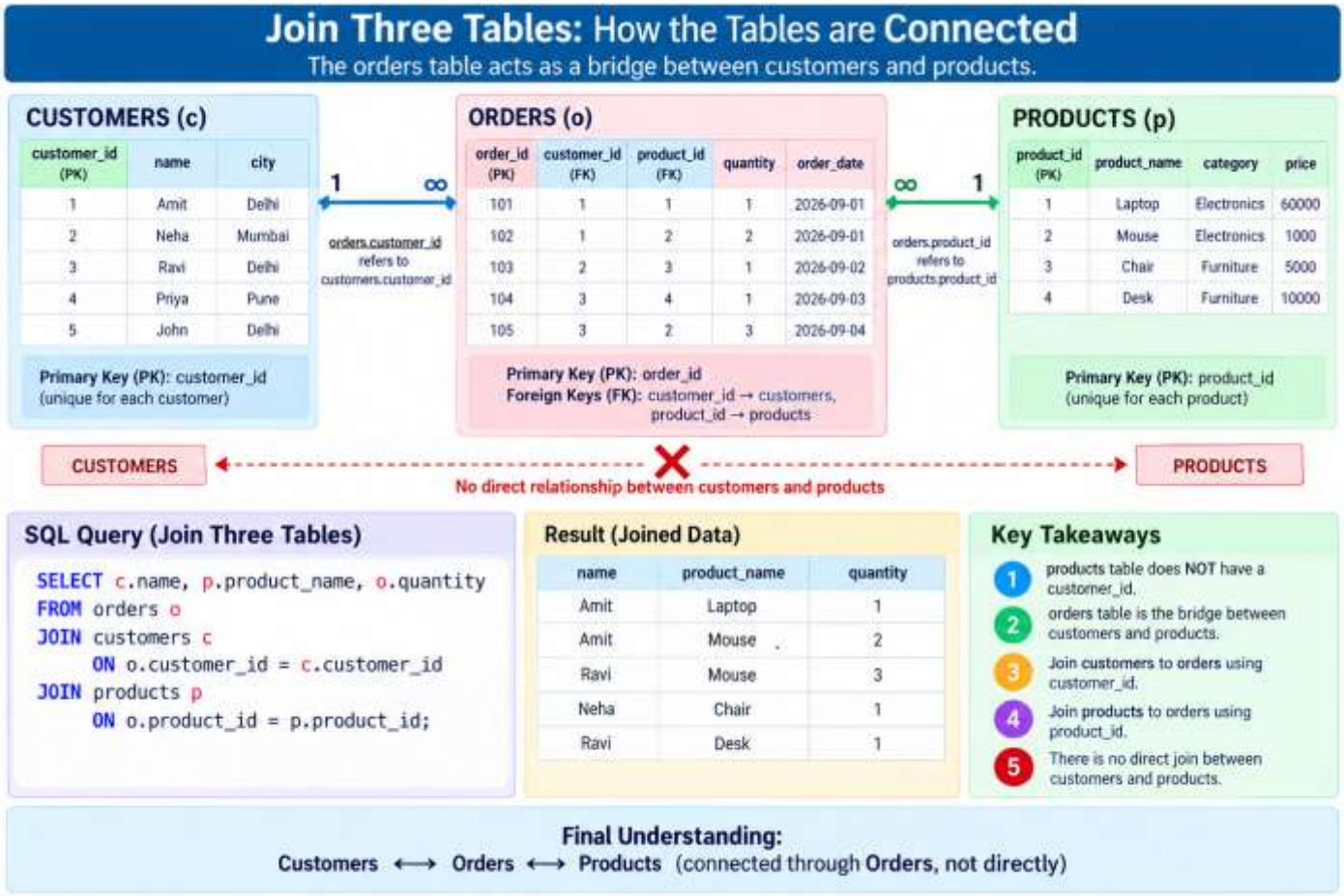
How it Works?

1. Join orders with customers using o.customer_id = c.customer_id (gets customer name for each order).
2. Then join the result with products using o.product_id = p.product_id (gets product name for each order).
3. Select the required columns (customer name, product name, quantity).

Key Takeaways

- You can join more than two tables in a single query.
- Use JOIN ... ON to specify how the tables are related.
- This helps to get a complete view by combining data from multiple tables.
- Very useful in real-world databases (e.g., customers, orders, products).

JOIN helps you combine related data from multiple tables to get meaningful information.



10. LEFT JOIN – Customers with No Orders ¶

9]:
```
pd.read_sql("""
SELECT c.name
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL
""", engine)
```

9]:

	name
0	Priya
1	John

11. UNION

0]:
```
pd.read_sql("""
SELECT city FROM customers
UNION
SELECT category FROM products
""", engine)
```

0]:

	city
0	Delhi
1	Mumbai
2	Pune
3	Electronics
4	Furniture

11. UNION in SQL

Combines the result sets of two SELECT statements and removes duplicate rows.

CUSTOMERS

(Table: customers)

customer_id	city
1	Delhi
2	Mumbai
3	Delhi
4	Pune

SELECT city
FROM customers

PRODUCTS

(Table: products)

product_id	category
1	Electronics
2	Electronics
3	Furniture
4	Furniture

SELECT category
FROM products

UNION

Combines both result sets and removes duplicate values

SQL Query

```
SELECT city
FROM customers
UNION
SELECT category
FROM products;
```

Result (Distinct Values)

	city
0	Delhi
1	Mumbai
2	Pune
3	Electronics
4	Furniture

Why Delhi appears once?

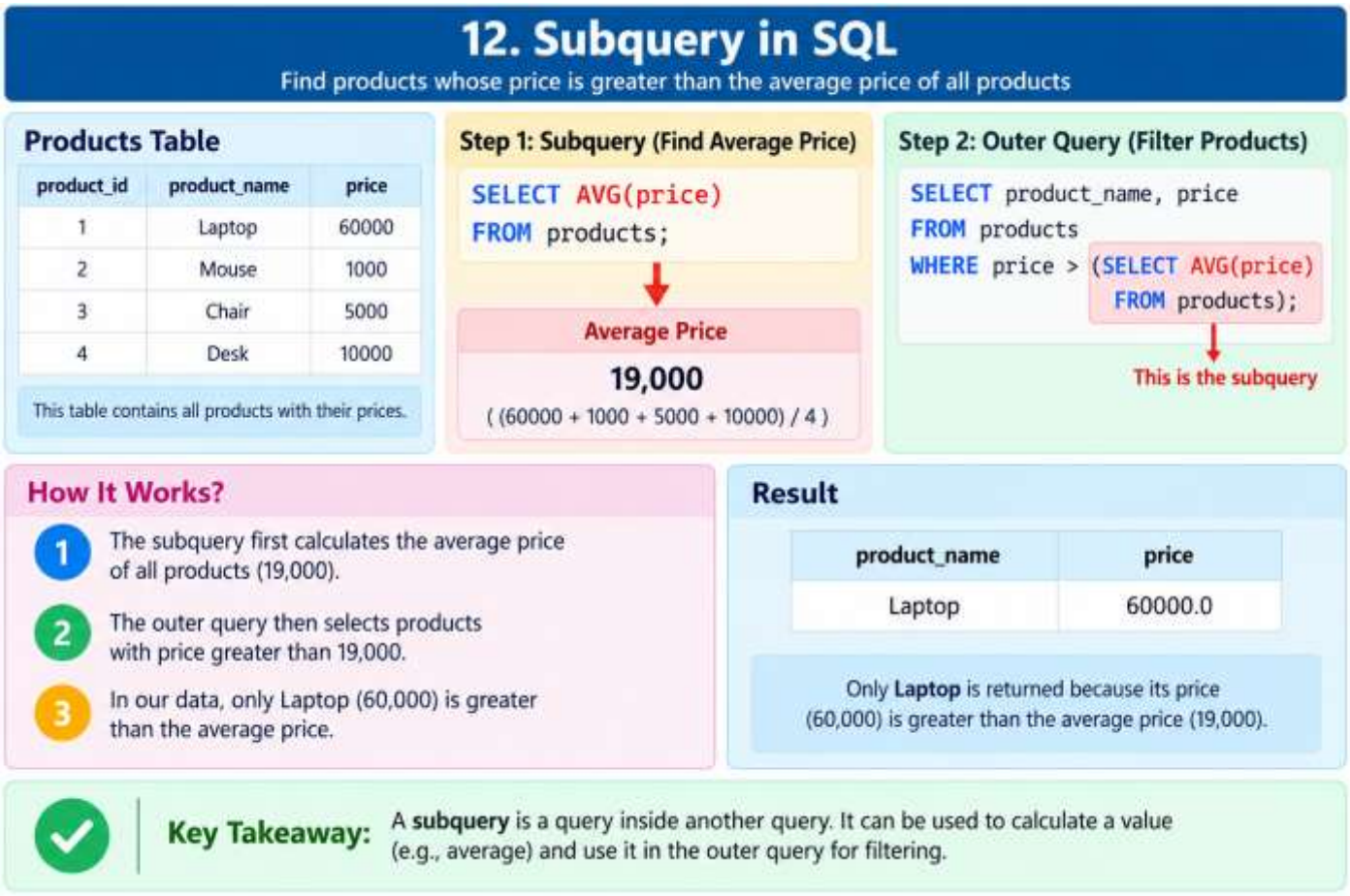
Delhi occurs twice in the customers table, but UNION removes duplicate values, so it appears only once in the final result.

Key Takeaway: **UNION** combines compatible SELECT results and removes duplicate values.

12. Subquery

```
pd.read_sql("""
SELECT product_name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products)
""", engine)
```

	product_name	price
0	Laptop	60000.0

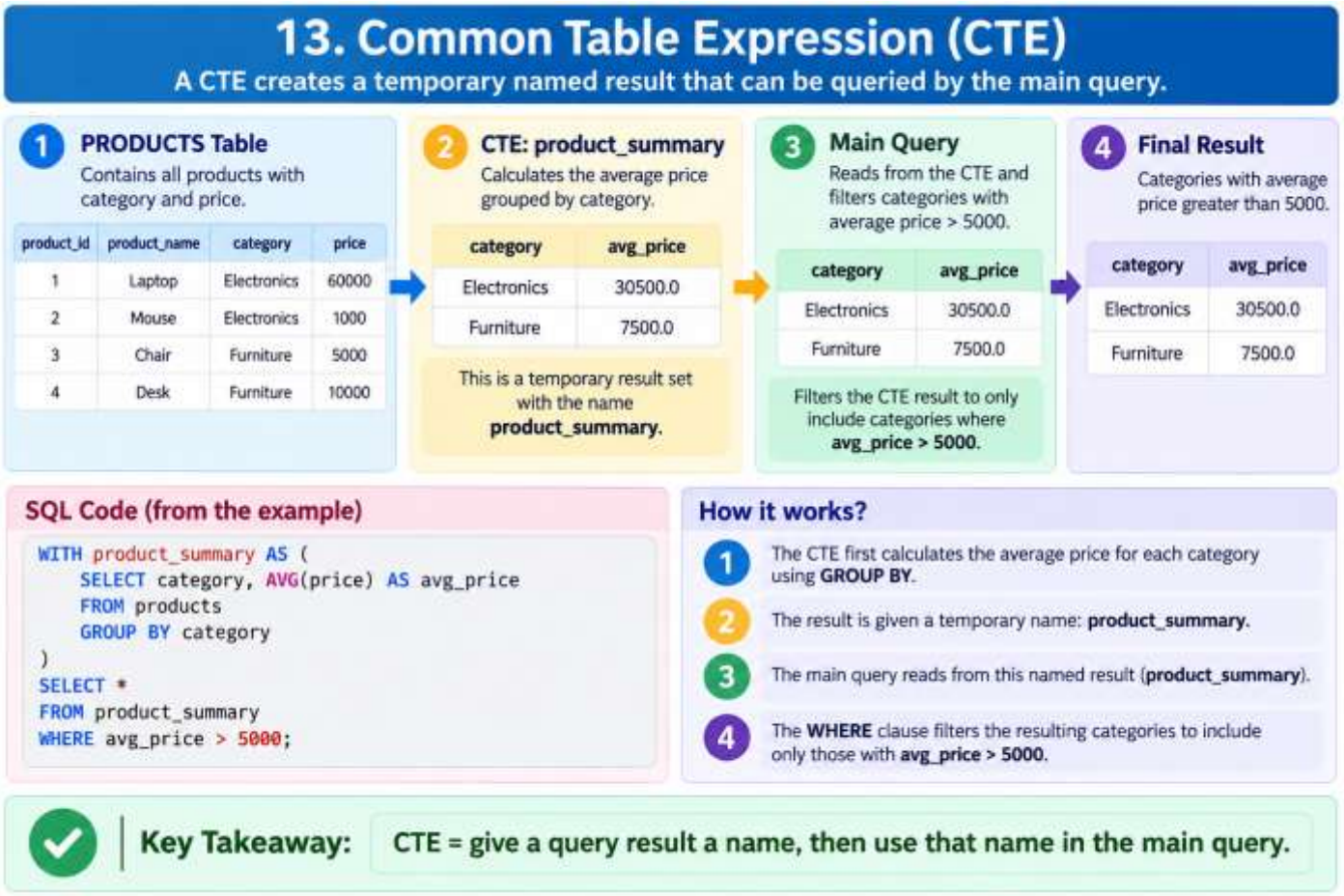


13. Common Table Expression (CTE)

```
]: pd.read_sql("""
WITH product_summary AS (
    SELECT category, AVG(price) AS avg_price
    FROM products
    GROUP BY category
)
SELECT *
FROM product_summary
WHERE avg_price > 5000
""", engine)
```

]:

	category	avg_price
0	Electronics	30500.0
1	Furniture	7500.0

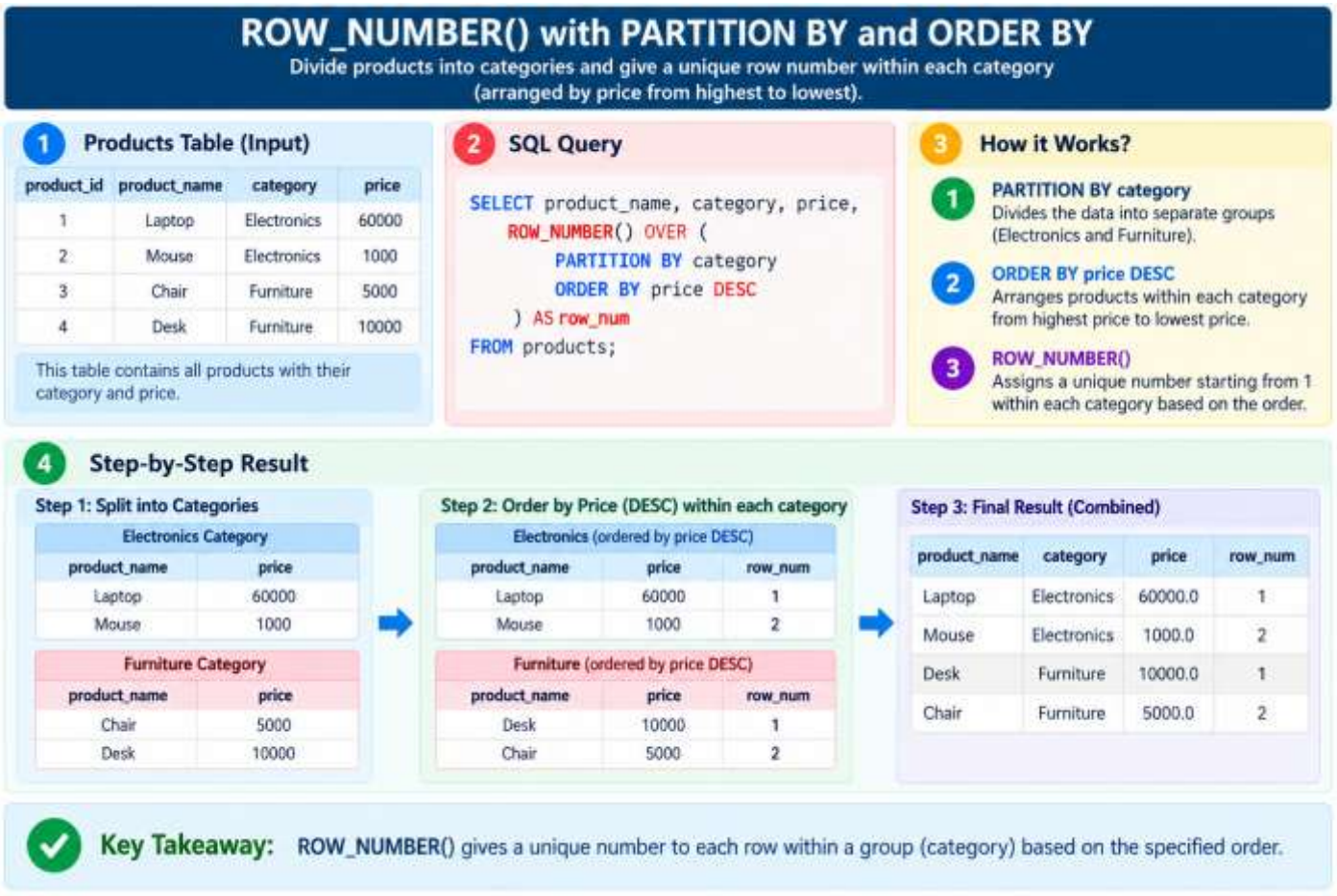


)

[53]:
```
# ROW_NUMBER
# ROW_NUMBER() gives a unique number to each row.
# Calculate the row number using the rules inside OVER().
#OVER() is what makes ROW_NUMBER() a window function.
#Divide the products into separate groups based on category.
#Within each category, arrange products from highest price to lowest price

pd.read_sql("""
SELECT product_name, category, price,
       ROW_NUMBER() OVER (
           PARTITION BY category
           ORDER BY price DESC
       ) AS row_num
FROM products
""", engine)
```

[53]:

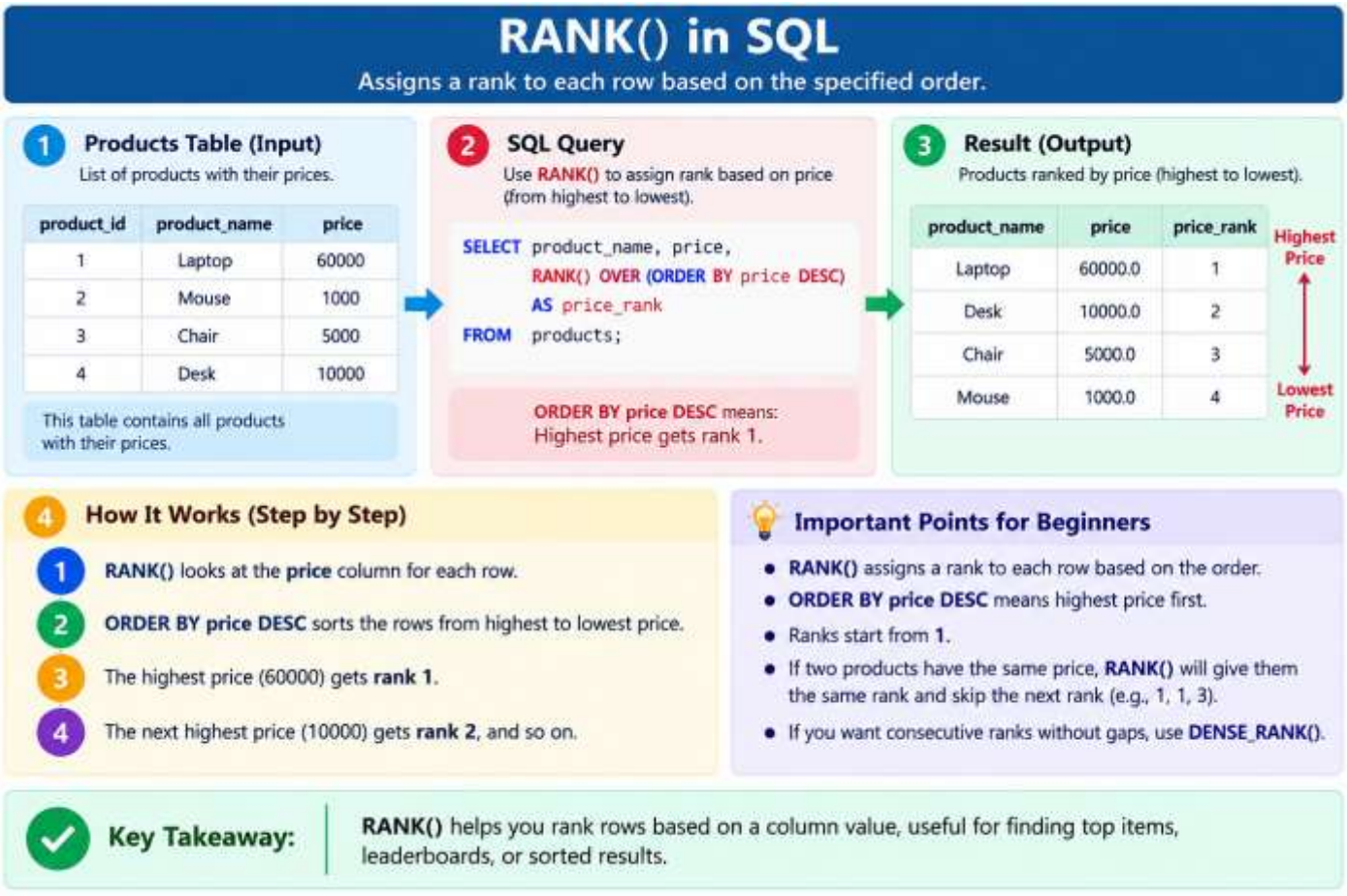
	product_name	category	price	row_num
0	Laptop	Electronics	60000.0	1
1	Mouse	Electronics	1000.0	2
2	Desk	Furniture	10000.0	1
3	Chair	Furniture	5000.0	2



[54]:
```
# RANK
pd.read_sql("""
SELECT product_name, price,
       RANK() OVER (ORDER BY price DESC) AS price_rank
FROM products
""", engine)
```

[54]:

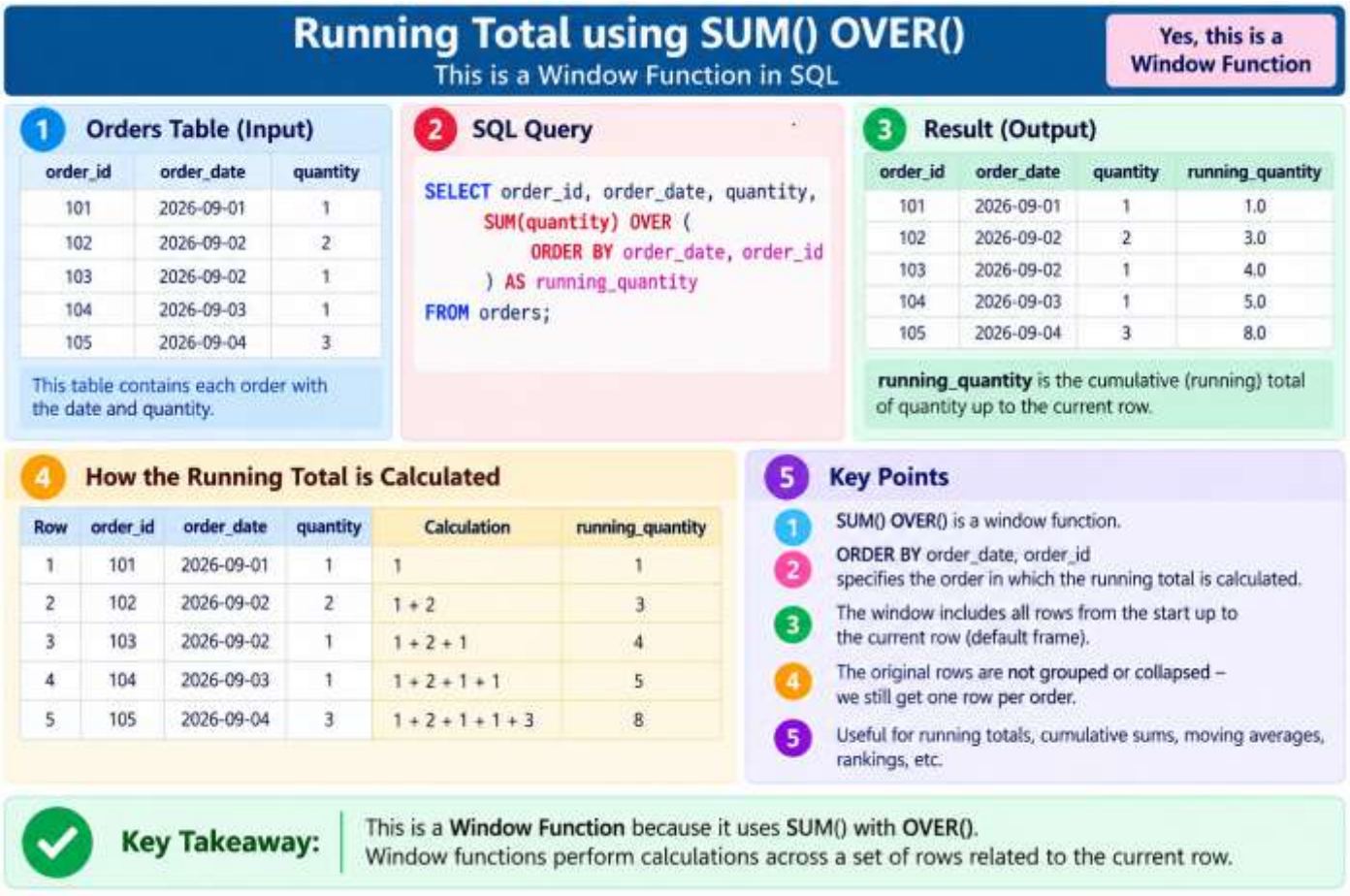
	product_name	price	price_rank
0	Laptop	60000.0	1
1	Desk	10000.0	2
2	Chair	5000.0	3
3	Mouse	1000.0	4



[55]:
```
# Running total
pd.read_sql("""
SELECT order_id, order_date, quantity,
       SUM(quantity) OVER (
           ORDER BY order_date, order_id
       ) AS running_quantity
FROM orders
""", engine)
```

[55]:

	order_id	order_date	quantity	running_quantity
0	101	2026-09-01	1	1.0
1	102	2026-09-02	2	3.0
2	103	2026-09-02	1	4.0
3	104	2026-09-03	1	5.0
4	105	2026-09-04	3	8.0



[56]:
```
# LAG
pd.read_sql("""
SELECT order_id, order_date, quantity,
       LAG(quantity) OVER (
           ORDER BY order_date, order_id
       ) AS previous_quantity
FROM orders
""", engine)
```

[56]:

	order_id	order_date	quantity	previous_quantity
0	101	2026-09-01	1	NaN
1	102	2026-09-02	2	1.0
2	103	2026-09-02	1	2.0
3	104	2026-09-03	1	1.0
4	105	2026-09-04	3	1.0

LAG() in SQL

LAG() gives the value from the previous row (based on the specified order).

1 Input Table: orders

order_id	order_date	quantity
101	2026-09-01	1
102	2026-09-02	2
103	2026-09-02	1
104	2026-09-03	1
105	2026-09-04	3

This table contains each order with the date and quantity.

2 SQL Query

```
SELECT order_id, order_date,
       quantity,
       LAG(quantity) OVER (
           ORDER BY order_date,
                    order_id
       ) AS previous_quantity
FROM orders;
```

3 How LAG() Works?

1. The rows are ordered by order_date, order_id.
2. For each row, LAG(quantity) looks at the quantity from the previous row.
3. The first row has no previous row, so it returns **NULL** (shown as NaN in pandas).
4. For all other rows, it shows the quantity from the row just above.

4 Step-by-Step Calculation

Row	order_id	order_date	quantity	Previous row quantity	previous_quantity (LAG)
1	101	2026-09-01	1	— (no previous row)	NULL (NaN)
2	102	2026-09-02	2	1	1
3	103	2026-09-02	1	2	2
4	104	2026-09-03	1	1	1
5	105	2026-09-04	3	1	1

First row has no previous value

Previous quantity = 1

Previous quantity = 2

Previous quantity = 1

Previous quantity = 1

5 Final Result (Output)

order_id	order_date	quantity	previous_quantity
101	2026-09-01	1	NaN
102	2026-09-02	2	1.0
103	2026-09-02	1	2.0
104	2026-09-03	1	1.0
105	2026-09-04	3	1.0

Key Takeaways

- **LAG()** is a window function.
- It gives the value from a previous row.
- The first row returns NULL because there is no previous row.
- Useful for comparing with previous values (e.g., day-over-day analysis).